

Reviewer Pack — Legendre-Fenchel Involution Conjecture: Discrepancy Invarian...

Entry 2f367d15ae54 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-26 21:43:02 UTC

Legendre-Fenchel Involution Conjecture: Discrepancy Invariance under Double Tropical Fourier Transform

Entry ID: 2f367d15ae54 **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-26 21:43:02 UTC

1. Conjecture

Field A (mathematical branch): TROPICAL_FOURIER_ANALYSIS framework (Tropical Geometry / Fourier-Analytic Combinatorics, max-plus semiring) **Field B** (complexity object): Query complexity of Legendre-Fenchel/tropical-Fourier inversion (number of TropicalFourierTransform oracle calls needed to recover a convex tropical polynomial)

Statement:

Let f be a convex TropicalPolynomial on a finite integer grid $X = \{0, 1, \dots, N-1\}$ with values in $\mathbb{R} \cup \{-\infty\}$, and let TFFT denote the max-plus TropicalFourierTransform defined by $\text{TFFT}(f)(k) = \max_{x \in X} \{f(x) - k \cdot x\}$ for $k \in X$. Then (i) $\text{TFFT}(\text{TFFT}(f))(x) = f(x)$ for all $x \in X$ (the involution form of axiom A2), and (ii) $\text{MinimalFourierCoefficient}(f) := \min_k \text{TFFT}(f)(k)$ satisfies $\text{MinimalFourierCoefficient}(\text{TFFT}(\text{TFFT}(f))) = \text{MinimalFourierCoefficient}(f)$, and moreover $\text{DiscrepancyMeasure}(f) \leq |\text{MinimalFourierCoefficient}(f)| + |\max_k \text{TFFT}(f)(k)|$ (consistent with axiom A3). In particular, the discrepancy bound is preserved under one round-trip through the tropical Fourier domain.

Rationale (proposer's reasoning):

This sub-conjecture directly stress-tests axiom A2 (invertibility of TropicalFourierTransform on a structured subclass) using the well-known fact that the Legendre-Fenchel transform — the natural max-plus analogue of the Fourier transform — is involutive on convex functions. Coupling A2 with axiom A3 implies that any quantity defined in terms of Fourier-coefficient extrema (including MinimalFourierCoefficient and the resulting Discrepancy bound) must be preserved under double TFFT. If A2 and A3 hold simultaneously on the convex class, this preservation must be exact; any failure would indicate that A2 fails outside trivial cases or that A3's Fourier-side bound is not tight on the involutive subclass.

Taxonomy category: FOURIER_ANALYTIC (status at proposal time:)

Framework membership: framework fw_28b4bfb95f, role: elaboration

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 11454e83a705d445

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 600 convex trials (200 each for $N \in \{16, 32, 64\}$), all three checks must hold: (a) $\max_x |TFT(TFT(f)) - f| < 1e-9$, (b) $|\text{MinFC}(TFT(TFT(f))) - \text{MinFC}(f)| < 1e-9$, (c) $\text{Discrepancy}(f) \leq |\min F| + |\max F|$. Non-convex control must show involution failure in ≥ 1 trial.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.97	SAFE	SAFE
ALGEBRIZATION	SAFE	0.95	SAFE	SAFE
KARP_LIPTON	SAFE	0.98	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - tropical Fourier transform Legendre-Fenchel involution convex - max-plus semiring Legendre transform discrepancy convex polynomial - tropical polynomial Legendre-Fenchel biconjugate query complexity oracle

Top relevant hits considered: - [http://arxiv.org/abs/1010.5964v1] Quadratic discrete Fourier transform and mutually unbiased bases - [http://arxiv.org/abs/1207.2443v2] Tropical Teichmüller and Siegel spaces - [http://arxiv.org/abs/2006.04823v3] Quantum Legendre-Fenchel Transform - [http://arxiv.org/abs/1712.03499v1] Linear regression over the max-plus semiring: algorithms and applications - [http://arxiv.org/abs/math/0208014v2] Rational semimodules over the max-plus semiring and geometric approach of discrete event systems - [http://arxiv.org/abs/2407.09775v1] Learning Weighted Finite Automata over the Max-Plus Semiring and its Termination - [http://arxiv.org/abs/1204.3875v1] Tropicalizing vs Compactifying the Torelli morphism - [http://arxiv.org/abs/0908.2468v1] Average/Worst-Case Gap of Quantum Query Complexities by On-Set Size

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
import json
from sys import argv

def max_plus(a, b):
    return float('-inf') if a == float('-inf') or b == float('-inf') else max(a, b)

def min_plus(*args):
    return float('inf') if all(x == float('inf') for x in args) else min(args)

def tropical_polynomial(N, slopes, offset):
```

```

f = [0] * N
current_slope = 0
for i in range(1, N):
    current_slope += random.uniform(-5, 5)
    f[i] = f[i-1] + current_slope
return [x + offset for x in f]

def tropical_fourier_transform(f, N):
    F = [float('-inf')] * N
    for k in range(N):
        for x in range(N):
            F[k] = max_plus(F[k], f[x] - k * x)
    return F

def run_trial(seed: int) -> dict:
    random.seed(seed)

    results = []
    for n in [5, 8, 11, 14]:
        N = 2 ** n
        for _ in range(200):
            slopes = sorted(random.uniform(-5, 5) for _ in range(N - 1))
            f = tropical_polynomial(N, slopes, random.uniform(-5, 5))
            F = tropical_fourier_transform(f, N)
            G = tropical_fourier_transform(F, N)

            max_diff = max(abs(G[x] - f[x]) for x in range(N))
            min_fc_f = min_plus(*F)
            min_fc_g = min_plus(*G)
            discrepancy_f = max_plus(*f) - min_plus(*f)
            discrepancy_g = max_plus(*G) - min_plus(*G)

            results.append({
                "n": n,
                "max_diff": max_diff,
                "min_fc_f": min_fc_f,
                "min_fc_g": min_fc_g,
                "discrepancy_f": discrepancy_f,
                "discrepancy_g": discrepancy_g
            })

    all_checks_passed = True
    for result in results:
        if result["max_diff"] >= 1e-9 or abs(result["min_fc_f"] - result["min_fc_g"]) >= 1e-9 or result["discrepancy_f"] >= 1e-9 or result["discrepancy_g"] >= 1e-9:
            all_checks_passed = False
            break

    return {
        "metric_name": "max_diff",
        "metric_value": sum(result["max_diff"] for result in results) / len(results),
        "instances_tested": len(results),
        "conjecture_holds": all_checks_passed,
        "counterexample": "" if all_checks_passed else "involution_failure"
    }

if __name__ == "__main__":

```

```

seeds = [int(x) for x in argv[1:] or [11, 23, 37, 53, 71]]

results = []
for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {json.dumps(result)}")
    results.append(result)

mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results) and sum(1 for r in results if not r["conjecture_holds"]) > 0:
    print(f"RESULT: FALSIFIED counterexample=\"involution_failure\" first_failing_seed={seeds[0]}")
else:
    print(f"RESULT: INCONCLUSIVE insufficient data to determine support of conjecture")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7303b4e9.py", line 85, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7303b4e9.py", line 54, in run_trial
    discrepancy_f = max_plus(*f) - min_plus(*f)
                      ~~~~~
TypeError: max_plus() takes 2 positional arguments but 32 were given

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a `TypeError` (`max_plus()` takes 2 positional arguments but 32 were given) before any trials produced data, so none of the pre-registered checks (a), (b), (c) were evaluated. | next: Fix the `max_plus/min_plus` invocation to fold over the list (e.g., `functools.reduce(max_plus, f)` or use Python's built-in `max/min` on the array) and rerun the 600 convex + non-convex control trials.

11. Audit log (LLM calls)

Total LLM calls: 6

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	preregistration	claude_max	opus	0	0	7449
2	novelty	claude_max	opus	0	0	3060
3	novelty	claude_max	opus	0	0	11374
4	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11785
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10385
6	judge	claude_max	opus	0	0	5400

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 49453 ms total latency. Provider mix: {'claude_max': 4, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/2f367d15ae54.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/2f367d15ae54.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/2f367d15ae54.tar.gz` (if generated)